

A Complete Semantics of CIRCT Core Dialects

Jianhong Zhao, Jinhui Kang

July 28, 2023

Table of Contents

- 1 Background
- 2 Goal & Challenges
- 3 Solution
- 4 Contributions
- 5 Preliminaries
- 6 Approach
- 7 Evaluation
- 8 Discussion

Situation: CIRCT is pretty!

- CIRCT (Circuit IR Compilers and Tools) is a novel infrastructure for building hardware compilers and tools.
- It aims to do for hardware **as LLVM did** for software via unified, consistent, and interoperable intermediate representation based on MLIR (Multi-Level Intermediate Representation) dialects.
- Since the performance, integration, and collaboration of CIRCT, novel hardware description languages, like **Chisel**, **Magma**, and **Moore**, use CIRCT to construct their compilers.

Problem: No Formalization! No Certainty!

- Towards a unified architecture to integrate the hardware tools, the **correctness of CIRCT is important**.
- And it is **worthwhile** to verify the hardware in CIRCT dialects as verify the software in LLVM IR.
- However, there is **no formal semantics** for CIRCT dialects, which means that there is **no certainty to prove** anything about CIRCT.

Goal & Challenges:

- Goal: Thus, we aim to construct the formal semantics for CIRCT dialects.
- Existing Efforts:
 - Intermediate Representation: LLVM, MLIR semantics
 - HDL: verilog, vhdل semantics
 - Semantics using K: C, Java ...
- Challenges in formalizing CIRCT
 - MLIR Based
 - Early Stage
 - Ambiguous Documentation
 - Semantics Validation

MLIR Based

- CIRCT is built on top of the LLVM compiler infrastructure and provides an intermediate representation based on MLIR dialects. Compared to traditional languages, MLIR dialects are **composable and unified** by the operation concept. This makes CIRCT a powerful tool for building hardware compilers and tools, as it allows for easy integration of different hardware description languages and transformations.
- However, this also makes CIRCT challenging to formalize, as the unified structure “operation” should have the **ability to capture arbitrary semantics** that other languages provide by different concepts, such as statements, expressions, and instructions. Moreover, the semantics of each dialect should be defined distinctly and composed in a **consistent manner**.

Early Stage

- CIRCT is a new project and is still in its infancy. Although it is already used by some novel languages and maintained by the LLVM community, **the flaws** in the documentation and **the modification** of the infrastructure are inevitable.
- This requires the formal semantics to be **flexible and extensible**. And the formal semantics should be able to **adapt to the changes** in CIRCT.

Ambiguous Documentation

- The flaws in the language reference are not the only problem. The CIRCT dialects explanation is **ambiguous and not enough**. For example, the *hw.module* operation even misses the syntax and semantics explanation. This requires the formal semantics to be able to fill the gaps in the documentation.

Semantics Validation

- The test in CIRCT is mainly focused on transformation and compilation, with **few semantic targeted tests** in the CIRCT test suite. Moreover, CIRCT provides intermediate representations rather than a language that can **perform simulations directly**. This makes it challenging to validate the formal semantics, along with the poor documentation and the early stage of CIRCT.

Any More? Any Incorrect?

- 你好

Solution Overview

- We present the first formal semantics of the **core dialects** in CIRCT, including **HW**, **Comb**, **Seq**, and **SV** dialects, since they are the **most stable** representations in the infrastructure and other dialects can **compile to** them.
- We employ the $\mathbb{K}$ framework to construct the semantics and generate various tools for it.

Solution Steps - 1

- ① We first formalize the core MLIR, including the syntax, preprocess of operations, and the builtin dialect, to provide a **solid and extendable foundation** for the semantics of the dialects and the consistent composition of them.
- ② Then we formalize the semantics of the CIRCT core dialects, including HW, Comb, Seq, and SV dialects. With the previous foundation, we only need to focus on the **distinctive features** of each dialect.
- ③ Next, we introduce a **C++ interface** to simulate the hardware, like verilator, based on the CIRCT semantics.
- ④ Finally, we propose a **benchmark and co-simulation** framework to end-to-end validate the semantics, by comparing the simulation result with verilator.

Solution Steps - 2: Benchmark

- 1 The benchmark includes the semantic **unit test for each operation** in the dialects to help the development of formal semantics and validate the correctness of each operation.
- 2 The benchmark also includes some **open-source hardware designs**, including Rocket, BOOM, and Reiscinator, from <https://github.com/circt/arc-tests> to validate the effectiveness of the semantic. (Optional BOOM, Reiscinator, and tests from verilator)

Contributions

- ① We introduce **K-CIRCT**, a framework for formalizing CIRCT dialects. It provides the core concepts and preprocesses of MLIR semantics, which is the foundation to formalize and compose the dialect semantics. With the framework, we can focus on the semantics of each dialect without concerning the relationship and consistency.
- ② We formalize the semantics of CIRCT core dialects, including HW, COMB, Seq, SV dialects. Since other CIRCT dialects can be compiled into the core dialects, this semantics is enough for real-world hardware (Rocket) simulation and verification.
- ③ We introduce a **C++ interface** to simulate the hardware, like verilator, based on the CIRCT semantics.
- ④ We propose a benchmark and co-simulation framework of CIRCT core dialects semantics. The benchmark includes the operation-level test cases for individual operations and the chip-level test cases for the whole semantics.

Preliminaries

- CIRCT
- K Framework

Approach Structure

- Overview
- Scope of the Work
- K-CIRCT
 - Formalization of Core MLIR
 - Common Library for CIRCT
- Formalization of Core CIRCT
 - Program Configuration
 - Semantics of Individual Operations (using a running example) (只有一个说明是否足够, HW、Comb、Seq、SV 是否有一些有特点的 Op 需要说明)
- Simulation Approach
 - Semantics for Simulation
 - C++ Interface for Stimulation

RQs

- **RQ.1 (Extendable Capability):** How can K-CIRCT be used to formalize the semantics of CIRCT dialects and what is the associated labor cost?
- **RQ.2 (Reliability):** To what extent can we trust the core semantics of CIRCT?
- **RQ.3 (Applications):** What are the applications of the formal semantics of CIRCT?

RQ.1 (Extendable Capability)

- a discussion of how to formalize the semantics of CIRCT dialects using K-CIRCT. 体现出每个 operation 可以独立进行定义，而不需要考虑其他的 operation。结构的优越性。库的完整性等。状态的可拓展性。
- 有关代码量的统计，可以对比 dialect feature 外的代码量和 dialect feature 的代码量。

RQ.2 (Reliability)

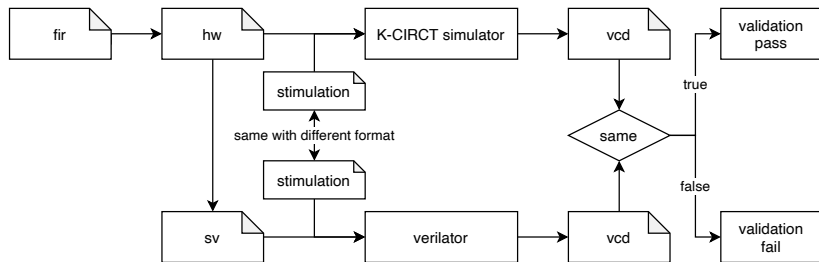


Figure 1: Co-simulation Framework

- benchmark setup
- co-simulation framework
- validation result discussion
- How many lines of code of the semantics? how intuitive and concise of the semantics?

RQ.3 (Applications)

- Simulation (Validation)
- Symbolic Simulation
- Hardware Verification
- Equivalence Verification
- Translation Validation

Discussion

- Limitations:
 - part of the dialects.
 - K framework 决定了我们语义的执行和验证效率。
- Future Work:
 - more dialects.
 - 执行与验证效率优化